

FORMULA1

Technical Execution Plan

Custom Gymnasium Racing Environment • Kaggle • PPO • Final Showdown

THE CORE CONCEPT

Every participant independently builds and trains a Reinforcement Learning racing agent using a standardized custom Gymnasium environment. Participants experiment with how their agent learns, while the organizers control the simulator, evaluation conditions, and Final Showdown pipeline.

Build an AI driver. Train it. Submit it. Watch it race.

PHASE 0

Lock the Technical Architecture

Standardize the infrastructure underneath while leaving room for meaningful experimentation.

Participants CAN modify

- Reward function
- Reward weights
- PPO hyperparameters
- Training timesteps
- Learning rate
- Gamma and batch size
- Selected training parameters

Participants CANNOT modify

- Environment API and core physics
- Track implementation
- Observation space structure
- Action space structure
- Neural network architecture
- RL algorithm
- Model export format
- Final evaluation environment

```
Same Environment
+
Same Observation Space
+
Same Action Space
+
Same PPO Architecture
↓
Different Reward Strategies
Different Training Strategies
↓
Different Driving Behaviour
```

WHY THIS MATTERS

Every submitted model follows the same underlying structure, so the organizer pipeline can load and evaluate all valid submissions consistently.

PHASE 1

Build the Custom Gymnasium Environment

The first technical milestone is a lightweight, deterministic 2D racing simulator designed specifically for Reinforcement Learning — not a high-fidelity automotive simulator.

```
formula1_env/
■■■ formula1_env.py
■■■ track.py
■■■ physics.py
■■■ renderer.py
■■■ config.py
```

The environment should follow the standard Gymnasium API.

```
class Formula1Env(gym.Env):  
  
    def reset(self, seed=None):  
        ...  
        return observation, info  
  
    def step(self, action):  
        ...  
        return observation, reward, terminated, truncated, info  
  
    def render(self):  
        ...
```

DESIGN PRIORITIES

Fast environment stepping, deterministic evaluation, stable training, clear behaviour, and easy rendering are more important than realistic vehicle physics.

PHASE 2

Design the Observation Space

Do not use raw image frames as the primary observation. A compact numerical state vector allows PPO to train efficiently with an MLP policy.

```
speed  
heading_error  
distance_from_track_center  
progress  
distance_to_left_edge  
distance_to_right_edge
```

Optionally, benchmark a small LiDAR-style sensor if it improves driving behaviour without making the environment unnecessarily heavy.

```
observation = [  
    *lidar_distances,  
    speed,  
    heading_error,  
    steering_angle  
]
```

TARGET

Keep the observation compact enough that participants can train, modify, and retrain multiple strategies during the workshop.

PHASE 3

Define the Action Space

Use a simple continuous control interface.

```

action_space = gym.spaces.Box(
    low=-1,
    high=1,
    shape=(2, )
)

```

- **Action[0] — Steering:** -1 full left, 0 straight, +1 full right
- **Action[1] — Throttle / Brake:** -1 brake, 0 coast, +1 accelerate

PHASE 4

Build the Baseline Reward Function

Reward engineering should be one of the main practical learning activities.

```

def calculate_reward(progress, speed, center_distance, crashed):
    reward = 0
    reward += progress * 10

    if crashed:
        reward -= 100

    return reward

```

- Speed incentives
- Track-centering penalties
- Smooth steering penalties
- Collision and off-track penalties
- Stable motion and racing-line rewards

WORKSHOP GOAL

Different reward strategies should create visibly different driving behaviour. That makes the RL concepts immediately visible during the Final Showdown.

PHASE 5

Train and Validate the Baseline Agent

Before building the participant workflow, prove that the environment can actually learn.

```

from stable_baselines3 import PPO

model = PPO(
    "MlpPolicy",
    env,
    verbose=1
)

model.learn(total_timesteps=...)

```

- Benchmark 50k, 100k, 250k, and 500k timesteps
- Record steps per second and total training duration

- Track average reward, progress, completion rate, and collision rate
- Visually inspect the agent at checkpoints

SUCCESS METRIC

The car visibly learns to navigate the track in a reasonable amount of time. This matters more than a pretty reward curve.

PHASE 6

Optimize the Workflow for Kaggle

Kaggle becomes the standardized participant development environment.

```
Participant Laptop
↓
Kaggle Notebook
↓
Custom Gymnasium Environment
↓
Stable-Baselines3
↓
PPO + MLP
↓
Training
```

- Test a fresh notebook and session
- Test all imports and dependencies
- Test environment rendering
- Test model saving and loading
- Test accelerator configuration
- Document any account preparation participants need

PHASE 7

Package the Workshop Repository

Keep the participant notebook separate from the underlying infrastructure.

```
formula1-workshop/
├── environment/
│   ├── formula1_env.py
│   ├── track.py
│   ├── physics.py
│   └── training/
│       ├── train.py
│       └── evaluation/
│           ├── evaluate.py
│           └── showcase/
│               ├── final_showdown.py
│               └── configs/
│                   ├── baseline.yaml
│                   └── notebooks/
│                       ├── Formula1_Workshop.ipynb
│                       └── README.md
```

FREEZE POINT

Once the environment is tested and validated, freeze the environment API before the event. The participant notebook should be the interface; the environment should be stable infrastructure.

PHASE 8

Build the Participant Kaggle Notebook

The notebook should guide participants through the RL workflow without drowning them in setup.

01	RL Introduction	Agent, environment, observation, action, reward, episode, and the RL loop.
02	Environment Setup	Initialize the provided environment with minimal setup.
03	Explore the Environment	Inspect observations/actions and visualize before training.
04	Train the Baseline	Train a known-working PPO baseline first.
05	Experiment Zone	Modify reward parameters and selected hyperparameters, then retrain and compare.

```
PROGRESS_WEIGHT = 10
SPEED_WEIGHT = 0.5
CENTERING_PENALTY = 2
CRASH_PENALTY = 100
```

PRACTICAL LOOP

Baseline → Modify → Retrain → Evaluate → Compare → Iterate

PHASE 9

Standardize Model Export

```
model.save(
    "/kaggle/working/formula1_model"
)
```

Expected model output: **formula1_model.zip**

```
submission/
■■■ formula1_model.zip
■■■ metadata.json
```

PHASE 10

Build the Submission Pipeline

```
Participant
↓
Export model
↓
Create submission ZIP
↓
Upload through submission form
↓
Organizer submission folder
↓
Automatic validation
```

PARTICIPANT RESPONSIBILITY

Train → Export → Submit. The participant should not need to understand the organizer-side showcase pipeline.

PHASE 11

Automatically Validate Every Submission

```
Submission received
↓
Extract files
↓
Validate structure
↓
Load model
↓
Create evaluation environment
↓
Run test rollout
↓
Check errors / NaNs
↓
VALID / DNF
```

FAILURE HANDLING

A broken or corrupted submission must never crash the entire showcase pipeline. Invalid models should be isolated and marked DNF.

PHASE 12

Run the Standardized Final Showdown

```

Load Model
↓
Fresh Environment
↓
Same Track
↓
Same Starting Position
↓
Same Environment Configuration
↓
Same Seed
↓
Fixed Evaluation Duration
↓
Capture Rollout

```

FAIRNESS RULE

Every model must experience equivalent evaluation conditions. The evaluation environment is controlled entirely by the organizers.

PHASE 13

Choose the Showcase Format

Initially, do not make participant models physically interact inside one shared simulation. Run each model independently and combine the rollouts visually.

MODEL A MODEL B MODEL C

■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
 ■ CAR ■ ■ CAR ■ ■ CAR ■
 ■ TRACK ■ ■ TRACK ■ ■ TRACK ■
 ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■

- Show large participant groups in rounds rather than one massive unreadable grid
- Optionally highlight categories such as Fastest, Most Consistent, Best Cornering, Best Reward Strategy, and Most Interesting Behaviour

PHASE 14

Pre-render the Final Showdown

```

Participant submits
↓
Validation
↓
Evaluation rollout
↓
Frame capture
↓
Video generation
↓
Mark submission READY

```

OPERATING PRINCIPLE

Process submissions as they arrive. The final minutes of the workshop should only require assembling already-generated rollouts rather than processing every model from scratch.

PHASE 15

Run the Full End-to-End Dry Run

```
Open Kaggle
↓
Open Starter Notebook
↓
Initialize Environment
↓
Run Random-Agent Demo
↓
Train Baseline PPO
↓
Modify Reward
↓
Retrain
↓
Evaluate
↓
Export Model
↓
Submit Model
↓
Validate Submission
↓
Generate Rollout
↓
Generate Final Video
↓
Play Final Video
```

DRY-RUN RULE

Use a machine and account that were not used during development. Time every stage and document every failure point before the actual event.

IMPLEMENTATION CHECKLIST

A practical build order for the organizing team.

PHASE 1 — ENVIRONMENT

- Build 2D track
- Build simplified car physics
- Implement Gymnasium API
- Define observation space
- Define action space
- Implement baseline reward
- Implement collision detection
- Implement track boundaries
- Implement termination conditions
- Implement rendering

PHASE 2 — RL VALIDATION

- Train PPO baseline
- Benchmark training speed
- Determine useful timestep budget
- Test reward modifications
- Verify visibly different behaviours
- Test multiple seeds
- Verify deterministic evaluation

PHASE 3 — KAGGLE

- Package environment
- Create starter notebook
- Test fresh session
- Test participant workflow
- Test model export
- Test model loading
- Document participant preparation

PHASE 4 — SUBMISSION

- Finalize submission format
- Build upload system
- Build validator
- Handle corrupted models
- Handle invalid submissions
- Handle missing files

- Generate DNF automatically

PHASE 5 — FINAL SHOWDOWN

- Build evaluation environment
- Fix evaluation seed
- Fix starting conditions
- Fix evaluation duration
- Build rollout renderer
- Build grid compositor
- Generate MP4
- Test large participant counts
- Prepare fallback model
- Test main-screen playback

FINAL RECOMMENDED STACK

PARTICIPANTS	ORGANIZERS
Kaggle Notebook	Submitted Model
↓	↓
Gymnasium	Validation
↓	↓
Custom Formula1 Environment	Standardized Evaluation
↓	↓
Stable-Baselines3	Fixed Seed + Fixed Track
↓	↓
PPO + MLP Policy	Rollout Generation
↓	↓
Reward + Hyperparameter Experiments	Video / Grid Composer
↓	↓
formula1_model.zip	FINAL SHOWDOWN

THE FIRST MILESTONE

Do not start with the Kaggle notebook. First build the custom Gymnasium racing environment and successfully train one PPO MLP agent to drive around the track. Once that works reliably, freeze the environment API and build everything else around it.

Custom Environment → PPO Baseline → Training Benchmark → Kaggle Notebook → Participant Experiments → Model Submission → Validation → Final Showdown